

Universidad Rafael Landívar.
Facultad de Ingeniería.
Ingeniería en Informática y Sistemas.
Laboratorio de Estructura de Datos I, Sección 01.
Docente: Ing. René Daniel Mejía Alvarado.

Proyecto de Aplicación No. 2
“Diccionario de Palabras”

Arrazola Escobar, Luna Samantha 1022725

Guatemala, 13 de mayo de 2026.

Tabla de contenido

1. Introducción
2. Planteamiento del problema, objetivos y alcance
 - 2.1 Problema a resolver
 - 2.2 Objetivo general
 - 2.3 Objetivos específicos
 - 2.4 Alcance del proyecto
3. Arquitectura general del sistema
4. Programación genérica, delegación e inyección de comportamiento
5. Modelado del dominio del problema
6. Explicación detallada de las estructuras implementadas
 - 6.1 Árbol de Prefijos (Trie)
 - 6.2 Tabla Hash con encadenamiento
 - 6.3 Cola de Prioridad basada en Heap
 - 6.4 Lista dinámica propia
 - 6.5 Persistencia CSV como soporte del sistema
7. API REST y persistencia de datos
8. Análisis formal de complejidad temporal y espacial
9. Buenas prácticas y decisiones de ingeniería aplicadas
10. Manual de funcionalidad del sistema
11. Conclusiones generales

1. Introducción

El proyecto de Diccionario de Palabras se desarrolló como una aplicación académica orientada a la integración de estructuras de datos avanzadas dentro de un contexto funcional y verificable mediante un API REST. El sistema permite almacenar palabras junto con su significado, buscar coincidencias exactas, realizar autocompletado por prefijo, ejecutar búsquedas por comodín, ordenar resultados por distintos criterios y conservar la información por medio de un archivo CSV.

La elección de un diccionario como problema de aplicación resulta pertinente para el curso de Estructura de Datos I porque combina varias necesidades algorítmicas diferentes. Una búsqueda exacta requiere acceso rápido por clave; una búsqueda por prefijo necesita recorrer caracteres de forma jerárquica; una búsqueda por comodín exige explorar rutas alternativas; y el ordenamiento de resultados necesita una estructura capaz de priorizar elementos de acuerdo con criterios configurables. Por esta razón, una única estructura de datos no es suficiente para resolver todo el problema de manera adecuada.

El sistema se construyó a partir de tres estructuras principales implementadas desde cero: un Árbol de Prefijos o Trie, una Tabla Hash con manejo de colisiones por encadenamiento y una Cola de Prioridad basada en Heap. Asimismo, se implementó una lista dinámica propia para evitar depender de ArrayList en la lógica interna del proyecto. Esta decisión responde directamente a la restricción del proyecto de no utilizar colecciones de Java ni librerías externas para resolver las estructuras solicitadas.

Desde la perspectiva de diseño, el proyecto también tiene programación genérica, delegación e inyección de comportamiento. Las estructuras fueron desarrolladas utilizando parámetros genéricos, Comparator, expresiones lambda e interfaces funcionales. Esto permitió separar la lógica interna de las estructuras de los criterios concretos de comparación. Por ejemplo, la cola de prioridad puede ordenar palabras por frecuencia de uso o alfabéticamente sin modificar su implementación interna.

El proyecto también se diferenció de una aplicación de consola tradicional porque fue construido como un API REST en Spring Boot. Esto permitió probar la funcionalidad mediante Postman, simulando solicitudes reales hacia endpoints como POST /palabra, GET /palabra/{valor}, GET /prefijo/{prefijo}, GET /comodin/{patron}, PUT /palabra y DELETE /palabra/{id}. De esta manera, el sistema no solo demuestra estructuras de datos, sino también su integración en una arquitectura moderna de software.

La persistencia se resolvió mediante un archivo CSV. Este archivo conserva el id, la palabra, el significado y la frecuencia de uso. Al iniciar el programa, el sistema lee el archivo y reconstruye sus estructuras internas; al crear, actualizar, eliminar o consultar palabras, el archivo se vuelve a guardar. Esto permite demostrar que el estado del diccionario se conserva entre ejecuciones sin necesidad de una base de datos externa.

El presente documento técnico describe el problema abordado, la arquitectura general del sistema, la organización de paquetes y clases, la explicación detallada de cada estructura implementada, el análisis formal de complejidad temporal y espacial, las buenas prácticas utilizadas y el manual de funcionalidad con pruebas desde Postman.

2. Planteamiento del problema, objetivos y alcance

2.1 Problema a resolver

Un diccionario digital debe responder de forma eficiente a varias operaciones comunes: registrar nuevas palabras, recuperar definiciones, buscar por coincidencias exactas, sugerir palabras a partir de un prefijo, localizar coincidencias parciales mediante comodines y ordenar los resultados obtenidos. Aunque estas operaciones parecen fáciles desde la perspectiva del usuario, internamente implican necesidades algorítmicas distintas.

El problema principal consiste en diseñar un sistema capaz de administrar palabras y significados sin depender de estructuras nativas ya resueltas como tal por el lenguaje. Esto implica construir manualmente mecanismos de almacenamiento, búsqueda, eliminación, actualización y ordenamiento. Asimismo, el sistema debe conservar sus datos entre ejecuciones mediante un archivo CSV, por lo que la información cargada debe sincronizarse correctamente con las estructuras internas.

La búsqueda exacta puede resolverse eficientemente mediante una tabla hash, ya que permite acceder a un valor a partir de una clave. Sin embargo, la tabla hash no es ideal para autocompletado porque no conserva relaciones entre prefijos. Para eso, se requiere un Trie, que almacena palabras carácter por carácter y permite recorrer prefijos de forma natural. Por otro lado, cuando los resultados deben limitarse y ordenarse por frecuencia o alfabéticamente, se requiere una cola de prioridad basada en heap.

2.2 Objetivo general

Desarrollar un API REST de un diccionario inteligente en Java y Spring Boot, utilizando estructuras de datos avanzadas implementadas desde cero, que permita almacenar, buscar, actualizar, eliminar, ordenar y persistir palabras mediante programación genérica, delegación e inyección de comportamiento.

2.3 Objetivos específicos

- Implementar un Árbol de Prefijos (Trie) genérico para almacenar palabras y permitir búsquedas por prefijo y comodín.
- Implementar una Tabla Hash genérica con manejo de colisiones por encadenamiento para búsquedas exactas por palabra y por ID.
- Implementar una Cola de Prioridad basada en Heap para obtener resultados ordenados y limitados por frecuencia o alfabéticamente.
- Construir una lista dinámica propia para almacenar resultados y evitar el uso de ArrayList como estructura principal.
- Aplicar Generics, Comparator, Comparable, expresiones lambda e interfaces funcionales para cumplir con programación genérica y delegación.
- Exponer la funcionalidad mediante endpoints HTTP utilizando Spring Boot.
- Implementar importación y exportación de datos mediante un archivo CSV.
- Conservar el estado del diccionario entre ejecuciones mediante persistencia en archivo.
- Documentar el funcionamiento del sistema y analizar formalmente la complejidad temporal y espacial de las operaciones principales.

2.4 Alcance del proyecto

El alcance del proyecto comprende la implementación funcional de un diccionario de palabras mediante un API REST. El sistema permite registrar palabras con su significado, consultar palabras por ID o coincidencia exacta, buscar palabras por prefijo, buscar mediante comodín, actualizar registros, eliminar palabras, ordenar resultados y limitar la cantidad de elementos devueltos.

También se incluye persistencia mediante un archivo CSV llamado `diccionario.csv`, ubicado dentro de la carpeta `data` del proyecto. Este archivo almacena los campos `id`, `palabra`, `significado` y `frecuencia`. Al iniciar la aplicación, el sistema reconstruye las estructuras internas a partir de dicho archivo. Al modificar el diccionario, el archivo se actualiza para conservar el estado. Asimismo, el uso de Spring Boot se limita a exponer la lógica del diccionario mediante endpoints REST que pueden probarse en Postman.

3. Arquitectura general del sistema

La arquitectura del sistema se diseñó con una separación clara entre capas. Esta organización permite distinguir entre la lógica del API, la lógica del diccionario, las estructuras de datos, los modelos de información y las utilidades de persistencia. Esta separación es importante porque facilita la explicación durante la defensa del proyecto y permite identificar con claridad qué responsabilidad cumple cada clase.

El paquete controller contiene las clases encargadas de recibir solicitudes HTTP. El paquete service concentra la lógica principal del sistema y funciona como punto de integración entre Trie, Tabla Hash, Heap, lista dinámica y CSV. El paquete model contiene los objetos que representan los datos del dominio. El paquete structure agrupa las estructuras implementadas desde cero y el paquete util contiene la clase encargada de leer y escribir el archivo CSV.

El flujo general del sistema inicia cuando el usuario envía una solicitud desde Postman. El controlador recibe la petición, extrae los parámetros o el cuerpo JSON y delega la operación al servicio. El servicio ejecuta la lógica correspondiente utilizando las estructuras internas y, cuando es necesario, actualiza el archivo CSV para conservar el estado.

Paquete	Clase	Responsabilidad principal
org.proyecto2	Main	Iniciar la aplicación Spring Boot y levantar el servidor embebido para exponer el API REST.
org.proyecto2.controller	PalabraController	Recibir solicitudes POST, PUT, GET y DELETE relacionadas con palabras del diccionario.
org.proyecto2.controller	CsvController	Exponer endpoints para importar y exportar información del diccionario mediante CSV.
org.proyecto2.service	DiccionarioService	Coordinar las operaciones del diccionario y sincronizar Trie, Hash, Heap, lista y CSV.
org.proyecto2.model	Palabra	Representar una palabra con id, texto, significado y frecuencia de uso.
org.proyecto2.model	PalabraRequest	Recibir datos de creación desde solicitudes HTTP sin exponer todos los atributos internos.
org.proyecto2.structure.common	CustomArrayList	Implementar una lista dinámica propia para almacenar resultados sin usar ArrayList.
org.proyecto2.structure.common	ExtractorClave	Definir una interfaz funcional para extraer claves de objetos genéricos.
org.proyecto2.structure.hash	CustomHashTable	Implementar una tabla hash genérica con colisiones por encadenamiento.
org.proyecto2.structure.hash	HashNode	Representar cada nodo enlazado dentro de los buckets de la tabla hash.
org.proyecto2.structure.trie	Trie	Implementar el árbol de prefijos para búsquedas por prefijo y comodín.
org.proyecto2.structure.trie	TrieNode	Representar cada nodo del Trie, sus hijos, dato asociado y marca de fin de palabra.

org.proyecto2.structure.priority	CustomPriorityQueue	Implementar una cola de prioridad basada en heap binario y Comparator.
org.proyecto2.util	CsvManager	Leer y escribir el archivo diccionario.csv para conservar el estado del sistema.

4. Programación genérica, delegación e inyección de comportamiento

4.1 Programación genérica

La programación genérica se usó como una decisión de diseño central. Las estructuras principales fueron construidas con parámetros de tipo, como T, K y V, para que pudieran trabajar con distintos tipos de datos sin quedar acopladas solo a la clase Palabra. Esta decisión mejora la reutilización del código y evidencia que las estructuras fueron diseñadas desde una perspectiva algorítmica general.

La Tabla Hash se implementó como CustomHashTable<K, V>, donde K representa la clave y V el valor asociado. Esto permite utilizar la misma clase para una tabla que indexa palabras por texto y otra que indexa palabras por ID. En el sistema se crean dos instancias: una con clave String y otra con clave Integer. Las dos comparten la misma implementación, pero trabajan con claves diferentes.

El Trie se implementó como Trie<T>. Esto significa que el árbol no almacena necesariamente cadenas simples, sino objetos genéricos. Para saber qué texto debe utilizar como clave, recibe una función externa. En este proyecto se utiliza con objetos Palabra, pero podría reutilizarse con cualquier otra entidad siempre que exista una manera de extraer su clave textual.

La Cola de Prioridad se implementó como CustomPriorityQueue<T>. El tipo de elemento no está fijado en el código. En el proyecto se usa con Palabra, pero la estructura podría ordenar otros objetos siempre que se le proporcione un Comparator adecuado.

4.2 Delegación de comportamiento

La delegación de comportamiento consiste en evitar que una estructura decida internamente todos los criterios de comparación o extracción de información. En lugar de escribir dentro del heap que las palabras deben ordenarse siempre por frecuencia o siempre alfabéticamente, el criterio se recibe desde afuera mediante Comparator.

Esto permite que el mismo heap se utilice para ordenar resultados por frecuencia de uso o por alfabeto. También permite invertir el orden entre ascendente y descendente sin reescribir la estructura. El heap solo se encarga de mantener la propiedad de prioridad; la definición de prioridad se delega al comparador recibido.

El mismo principio aparece en el Trie mediante la interfaz ExtractorClave. El Trie no sabe por sí mismo cuál atributo de Palabra debe usar para construir el camino de caracteres. Esa decisión se delega a una expresión lambda que obtiene la palabra desde el objeto.

4.3 Expresiones lambda

Las expresiones lambda se utilizaron para escribir comportamientos pequeños de manera clara. Por ejemplo, para crear el Trie se inyecta una lambda que indica cómo extraer el texto de la palabra. Esta lambda representa una función que recibe un objeto Palabra y retorna su atributo palabra.

También se utilizaron lambdas para construir comparadores. El comparador por frecuencia compara dos objetos Palabra utilizando getFrecuencia(), mientras que el comparador alfabético compara utilizando getPalabra().compareTo(). Así, el criterio de ordenamiento se define de forma compacta, pero sigue siendo explícito y defendible.

4.4 Inyección de comportamiento

La inyección de comportamiento se evidencia cuando una clase recibe desde afuera la lógica que necesita para operar. En el proyecto, el Trie recibe la forma de extraer una clave textual y la cola de prioridad recibe el criterio de comparación. Esto evita que las estructuras dependan de una única entidad o de una única forma de ordenar.

Esta decisión fortalece la calidad del diseño porque permite extender el sistema con nuevos criterios sin modificar las estructuras. Si en el futuro se quisiera ordenar por longitud de palabra, bastaría con crear un nuevo Comparador. El heap no tendría que cambiar su código interno.

Esta parte demuestra que el proyecto no solo implementa estructuras, sino que las implementa con flexibilidad y separación entre algoritmo y política de uso.

5. Modelado del dominio del problema

5.1 Palabra como entidad principal

La clase Palabra representa el elemento central del diccionario. Cada palabra almacenada en el sistema se modela como un objeto con cuatro atributos: id, palabra, significado y frecuencia. Estos atributos permiten identificar, describir y medir el uso de cada entrada del diccionario.

El id funciona como identificador único secuencial. Este valor es generado automáticamente por el sistema al crear una nueva palabra. Su función principal es permitir búsquedas directas por identificador y mantener consistencia durante actualizaciones o eliminaciones.

El atributo palabra almacena el texto principal que será indexado en el Trie y en la tabla hash por palabra. El atributo significado guarda la definición asociada. La frecuencia registra cuántas veces ha sido consultada una palabra mediante búsqueda exacta o por ID.

La clase Palabra implementa Comparable<Palabra>, lo cual permite definir un orden natural. En este proyecto, el orden natural se puede interpretar de manera alfabética a partir del texto de la palabra. Aunque el sistema utiliza principalmente Comparador para criterios dinámicos, Comparable permite cumplir con el requisito de ordenamiento por orden natural.

5.2 PalabraRequest como objeto de transferencia

PalabraRequest es una clase auxiliar utilizada para recibir datos desde solicitudes POST. Su objetivo es separar el cuerpo de la solicitud del modelo completo Palabra. Al crear una palabra, el usuario solo debe proporcionar el texto y el significado; el id y la frecuencia se generan internamente.

Esta separación mejora el diseño del API porque evita que el usuario tenga que enviar información que no le corresponde asignar. También permite validar con mayor claridad los datos de entrada.

5.3 Frecuencia como elemento de comportamiento

La frecuencia de uso es un atributo importante porque convierte al diccionario en un sistema que no solo almacena definiciones, sino que también registra interacción. Cada vez que una palabra se consulta por ID o por coincidencia exacta, la frecuencia aumenta.

Este valor se utiliza posteriormente como criterio de ordenamiento. De esta manera, el sistema puede devolver primero las palabras más consultadas o las menos consultadas, según el parámetro solicitado en el endpoint.

6. Explicación detallada de las estructuras implementadas

6.1 Árbol de Prefijos (Trie)

El Trie es una estructura de datos especialmente diseñada para almacenar cadenas de texto. Su funcionamiento se basa en representar cada palabra como una ruta de caracteres desde una raíz hasta un nodo final. Cada nodo representa una letra y puede tener varios hijos asociados a los caracteres siguientes.

En el proyecto, el Trie se implementó de forma genérica como `Trie<T>`. Esto significa que no almacena solo cadenas, sino que objetos completos. En este caso almacena objetos `Palabra`. Para saber qué texto debe usar como ruta, el Trie recibe un `ExtractorClave<T, String>`, que permite obtener la palabra desde el objeto.

La inserción inicia en la raíz. Por cada carácter de la palabra, se verifica si existe el hijo correspondiente. Si no existe, se crea un nuevo nodo. Al llegar al último carácter, el nodo se marca como fin de palabra y se almacena el objeto completo.

La búsqueda por prefijo consiste en recorrer los caracteres del prefijo. Si el camino existe, el algoritmo recolecta todas las palabras que se encuentran debajo de ese nodo. Esta operación es eficiente porque no necesita revisar palabras que no comparten el prefijo solicitado.

La búsqueda por comodín utiliza recursividad. Cuando el patrón contiene un asterisco, el algoritmo considera que dicho símbolo puede representar cero o más caracteres. Por eso, explora varias rutas posibles dentro del árbol. Esta operación puede ser más costosa que la búsqueda por prefijo, pero permite resolver patrones flexibles como `h*!*`.

La eliminación de una palabra requiere quitar la marca de fin de palabra y limpiar nodos que ya no sean necesarios. Este proceso debe hacerse con cuidado para no eliminar prefijos compartidos por otras palabras. Por ejemplo, si se elimina `hola`, pero existe `horno`, el nodo `h` y el nodo `o` no deben eliminarse si siguen siendo utilizados por otra ruta.

6.2 Tabla Hash con encadenamiento

La Tabla Hash permite asociar una clave con un valor. En el proyecto se implementó como `CustomHashTable<K, V>`, lo que permite utilizar distintos tipos de clave y valor. Esta estructura se utiliza dos veces: una para buscar palabras por texto y otra para buscar por ID.

La posición de almacenamiento se calcula utilizando el método `hashCode` de la clave y aplicando módulo sobre la capacidad interna de la tabla. Esta operación produce un índice dentro del arreglo de buckets.

Como diferentes claves pueden producir el mismo índice, se implementó manejo de colisiones por encadenamiento. Cada posición de la tabla puede contener una lista enlazada de nodos `HashNode`. Si una nueva clave cae en una posición ocupada, se recorre la cadena hasta encontrar la clave o insertar el nuevo nodo al final.

La búsqueda exacta recorre solo el bucket calculado. En promedio, si la distribución de claves es adecuada, la búsqueda es constante. En el peor caso, si muchas claves caen en el mismo bucket, la operación puede volverse lineal.

La actualización se resuelve insertando nuevamente una clave existente con un valor actualizado. La eliminación requiere localizar el nodo dentro de la cadena y ajustar las referencias del nodo anterior para saltar el nodo eliminado.

6.3 Cola de Prioridad basada en Heap

La Cola de Prioridad se implementó mediante un heap binario almacenado en un arreglo interno. El heap es una estructura que permite mantener siempre en la raíz el elemento de mayor prioridad según un criterio de comparación.

En el proyecto, la prioridad no está fija. El criterio se recibe mediante `Comparator<T>`. Esto permite que la misma estructura ordene palabras por frecuencia o alfabéticamente y que también pueda devolver resultados en orden ascendente o descendente.

La inserción agrega el nuevo elemento al final del arreglo y luego ejecuta un proceso de subida. Durante la subida, el elemento se compara con su padre y se intercambia si tiene mayor prioridad. Este proceso continúa hasta que el elemento queda en una posición válida.

La extracción toma el elemento de la raíz, mueve el último elemento a esa posición y ejecuta un proceso de bajada. Durante la bajada, el elemento se compara con sus hijos y se intercambia con el hijo de mayor prioridad hasta restaurar la propiedad de heap.

Esta estructura se utiliza para obtener el Top-K de resultados. Primero se insertan en el heap las palabras encontradas por prefijo o comodín; luego se extraen los primeros k elementos según el límite indicado por el usuario.

6.4 Lista dinámica propia

`CustomArrayList` es una estructura auxiliar implementada para evitar el uso de `ArrayList` en la lógica principal. Internamente utiliza un arreglo de `Object` que crece cuando se llena. Permite agregar, obtener, actualizar, eliminar y convertir los elementos a un arreglo simple para responder desde el API.

Esta estructura no reemplaza directamente a las estructuras obligatorias, pero es necesaria para almacenar resultados temporales y mantener consistencia con la restricción de no usar colecciones nativas para la lógica interna.

6.5 Persistencia CSV como soporte del sistema

`CsvManager` no es una estructura de datos principal, pero es importante para cumplir con la persistencia. Esta clase se encarga de leer el archivo `diccionario.csv` al iniciar el sistema y escribir el estado actualizado cuando ocurren cambios.

El archivo CSV contiene un encabezado con los campos `id`, `palabra`, `significado` y `frecuencia`. Cada línea posterior representa una palabra registrada. Esta representación facilita revisar manualmente los datos y demostrar que la información persiste entre ejecuciones.

7. API REST y persistencia de datos

El proyecto utiliza Spring Boot para exponer la funcionalidad del diccionario mediante un API REST. Spring Boot permite levantar un servidor embebido y recibir solicitudes HTTP sin configurar manualmente un servidor externo. En este sistema, la clase Main inicia la aplicación y permite que los controladores reciban solicitudes desde Postman.

Los controladores no contienen la lógica algorítmica de las estructuras. Su función principal es recibir parámetros, cuerpos JSON y rutas, para luego delegar el procesamiento hacia DiccionarioService. Esta separación permite que la lógica de estructuras permanezca concentrada en el servicio y no se mezcle con detalles HTTP.

Método	Endpoint	Función	Estructuras involucradas
POST	/palabra	Crear una nueva palabra con significado y frecuencia inicial.	Hash, Trie, lista, CSV
PUT	/palabra	Actualizar palabra, significado o frecuencia.	Hash, Trie, lista, CSV
GET	/palabra/{valor}	Buscar por ID si el valor es numérico o por palabra exacta si es texto.	Hash por ID o Hash por palabra
GET	/prefijo/{prefijo}	Buscar palabras que comienzan con un prefijo.	Trie y Heap
GET	/comodin/{patron}	Buscar coincidencias usando el comodín *.	Trie y Heap
DELETE	/palabra/{id}	Eliminar una palabra existente.	Hash, Trie, lista, CSV
POST	/csv/importar	Recargar el diccionario desde el archivo CSV.	CSV, Hash, Trie, lista
GET	/csv/exportar	Guardar el estado actual en el archivo CSV.	Lista y CSV

7.1 Persistencia CSV

La persistencia CSV permite conservar el estado del sistema entre ejecuciones. Cada palabra se almacena en una fila del archivo diccionario.csv. El encabezado id,palabra,significado,frecuencia permite identificar los atributos que se deben reconstruir al cargar el sistema.

Cuando se crea una palabra mediante POST, el servicio asigna un ID secuencial, inserta la palabra en las estructuras y guarda el archivo CSV. Cuando se actualiza o elimina una palabra, también se actualiza el archivo. Cuando se realiza una búsqueda exacta, la frecuencia cambia y por eso también se guarda el estado actualizado.

7.2 Proceso al iniciar el sistema

1. La clase Main inicia Spring Boot y levanta el servidor.
2. Spring crea una instancia de DicionarioService.
3. El constructor del servicio inicializa Trie, Hash por palabra, Hash por ID y lista propia.
4. El servicio llama a CsvManager para leer diccionario.csv.
5. Cada línea válida del archivo se convierte en un objeto Palabra.
6. Cada palabra se inserta en Trie, Hash por palabra, Hash por ID y lista.
7. El sistema calcula el mayor ID encontrado para determinar el siguiente ID disponible.

Este proceso evidencia que el estado del sistema se conserva entre ejecuciones. Si el programa se cierra y se vuelve a ejecutar, las palabras previamente guardadas continúan disponibles porque se reconstruyen desde el archivo CSV.

8. Análisis formal de complejidad temporal y espacial

En esta sección se presenta el análisis formal de complejidad temporal y espacial de las estructuras implementadas. Se utilizan las notaciones Ω para mejor caso, Θ para caso promedio y O para peor caso. Además de las tablas, se incluye una explicación breve para relacionar los valores con la implementación del proyecto.

8.1 Complejidad del Trie

Tabla 1. Complejidad temporal del Trie

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Insertar	La palabra es vacía o de longitud mínima	$\Omega(1)$	Se recorre cada carácter	$\Theta(m)$	Palabra de longitud m	$O(m)$
Buscar exacto	Coincidencia inmediata o texto mínimo	$\Omega(1)$	Se recorre la palabra	$\Theta(m)$	Camino completo hasta fallar o encontrar	$O(m)$
Buscar por prefijo	Prefijo vacío o muy corto	$\Omega(1)$	Se recorre el prefijo y se recolectan resultados	$\Theta(p+r)$	Prefijo amplio con muchos resultados	$O(p+r)$
Buscar comodín	Patrón con pocas ramas	$\Omega(m)$	Se exploran varias rutas	$\Theta(b^a)$	Comodines generan muchas combinaciones	$O(b^a)$
Eliminar	Palabra inexistente al inicio	$\Omega(1)$	Se recorre la palabra	$\Theta(m)$	Se limpian nodos hasta la raíz	$O(m)$

En el Trie, m representa la longitud de la palabra, p la longitud del prefijo, r la cantidad de resultados recolectados, b el factor de ramificación y a la cantidad de comodines. La búsqueda por prefijo es eficiente porque descarta todas las palabras que no comparten la ruta inicial. En cambio, la búsqueda con comodín puede ser más costosa porque debe explorar rutas alternativas.

Tabla 2. Complejidad espacial del Trie

Elemento	Descripción	Complejidad
Nodos del Trie	Cada carácter puede generar un nodo	$O(n*m)$
Dato asociado	Cada palabra completa almacena referencia al objeto	$O(n)$
Pila recursiva	Usada en comodín y eliminación	$O(m)$ a $O(h)$

El espacio del Trie depende del número de palabras y de su longitud. Sin embargo, cuando varias palabras comparten prefijos, se reutilizan nodos y el uso real de memoria puede ser menor que almacenar cada palabra de forma completamente independiente.

8.2 Complejidad de la Tabla Hash

Tabla 3. Complejidad temporal de la Tabla Hash

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Insertar	Bucket vacío	$\Omega(1)$	Colisiones controladas	$\Theta(1)$	Todas las claves en un bucket	$O(n)$
Buscar	Clave al inicio del bucket	$\Omega(1)$	Distribución uniforme	$\Theta(1)$	Recorrer cadena completa	$O(n)$
Actualizar	Clave ubicada rápidamente	$\Omega(1)$	Colisiones controladas	$\Theta(1)$	Buscar en cadena larga	$O(n)$
Eliminar	Clave en primer nodo	$\Omega(1)$	Recorrido corto	$\Theta(1)$	Clave al final de cadena	$O(n)$

La Tabla Hash tiene un rendimiento promedio constante cuando la función hash distribuye adecuadamente las claves. El peor caso ocurre cuando muchas claves colisionan en el mismo bucket, convirtiendo la búsqueda en un recorrido lineal por la cadena enlazada.

Tabla 4. Complejidad espacial de la Tabla Hash

Elemento	Descripción	Complejidad
Arreglo de buckets	Estructura base de la tabla	$O(c)$
Nodos enlazados	Un nodo por elemento insertado	$O(n)$
Referencias auxiliares	Variables temporales de recorrido	$O(1)$

La tabla hash requiere espacio para el arreglo de buckets y para los nodos almacenados. En términos asintóticos, si la capacidad crece proporcionalmente a los elementos, el espacio total se considera $O(n)$.

8.3 Complejidad del Heap

Tabla 5. Complejidad temporal del Heap

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Insertar	No requiere subir	$\Omega(1)$	Sube algunos niveles	$\Theta(\log n)$	Sube hasta la raíz	$O(\log n)$
Extraer	Reacomodo mínimo	$\Omega(1)$	Baja algunos niveles	$\Theta(\log n)$	Baja hasta una hoja	$O(\log n)$
Ver primero	Consultar raíz	$\Omega(1)$	Consultar raíz	$\Theta(1)$	Consultar raíz	$O(1)$

Extraer top K	K pequeño	$\Omega(k)$	Extrae k elementos	$\Theta(k \log n)$	Extrae k con reacomodo completo	$O(k \log n)$
---------------	-----------	-------------	--------------------	--------------------	---------------------------------	---------------

El heap mantiene una forma casi completa, por lo que su altura es logarítmica. Por esta razón, insertar y extraer tienen costo $O(\log n)$. Para obtener top K, se realizan k extracciones, cada una con costo logarítmico.

Tabla 6. Complejidad espacial del Heap

Elemento	Descripción	Complejidad
Arreglo interno	Almacena todos los elementos del heap	$O(n)$
Variables de intercambio	Usadas en subir y bajar	$O(1)$
Lista de resultados	Al devolver top K	$O(k)$

El heap almacena todos sus elementos en un arreglo interno. El proceso de reacomodo utiliza pocas variables auxiliares, por lo que el espacio adicional es constante, excepto cuando se genera una lista de resultados.

8.4 Complejidad de CustomArrayList

Tabla 7. Complejidad temporal de CustomArrayList

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Agregar	Hay espacio disponible	$\Omega(1)$	Inserción al final	$\Theta(1)$	Crece y copia elementos	$O(n)$
Obtener	Acceso directo	$\Omega(1)$	Acceso directo	$\Theta(1)$	Acceso directo	$O(1)$
Eliminar	Último elemento	$\Omega(1)$	Desplaza parte de la lista	$\Theta(n)$	Elimina el primero y desplaza todo	$O(n)$

La lista dinámica propia permite acceso directo por índice. Agregar es constante en promedio, pero cuando el arreglo interno se llena, debe crear uno nuevo y copiar los elementos existentes.

Tabla 8. Complejidad espacial de CustomArrayList

Elemento	Descripción	Complejidad
Arreglo interno	Reserva capacidad para los elementos	$O(n)$
Copia al crecer	Arreglo temporal durante expansión	$O(n)$
Variables auxiliares	Índices y temporales	$O(1)$

La lista dinámica utiliza espacio proporcional a la cantidad de elementos almacenados. Durante una expansión puede existir temporalmente un segundo arreglo, pero el orden espacial se mantiene lineal.

8.5 Complejidad de la persistencia CSV

Tabla 9. Complejidad temporal de la persistencia CSV

Operación	Mejor caso	Ω	Promedio	Θ	Peor caso	O
Importar CSV	Archivo vacío	$\Omega(1)$	Leer n líneas	$\Theta(n)$	Leer n líneas y convertir objetos	$O(n)$
Exportar CSV	Lista vacía	$\Omega(1)$	Escribir n palabras	$\Theta(n)$	Escribir n palabras completas	$O(n)$
Actualizar estado	Guardar lista actual	$\Omega(n)$	Guardar lista actual	$\Theta(n)$	Guardar lista completa	$O(n)$

La persistencia CSV es lineal porque debe leer o escribir cada palabra del diccionario. Aunque no es la operación más eficiente posible, es adecuada para el alcance académico del proyecto y permite demostrar conservación de estado sin bases de datos externas.

Tabla 10. Complejidad espacial del CSV

Elemento	Descripción	Complejidad
Lectura de archivo	Palabras cargadas a lista temporal	$O(n)$
Escritura de archivo	Escritura línea por línea	$O(1)$ adicional
Reconstrucción de estructuras	Inserta datos en Trie, Hash y lista	$O(n)$ más estructuras

Al importar, el sistema puede requerir espacio temporal para las palabras leídas antes de insertarlas en las estructuras. Al exportar, la escritura puede realizarse línea por línea con espacio adicional constante.

9. Buenas prácticas y decisiones de ingeniería aplicadas

Antes de presentar las conclusiones, es importante mencionar las buenas prácticas empleadas durante la construcción del proyecto. La robustez de una solución no depende solo de que las estructuras funcionen, sino también de cómo se organizan, cómo se protegen ante errores y cómo se preparan para ser explicadas después.

En primer lugar, se aplicó separación de responsabilidades. Las estructuras de datos no conocen detalles de HTTP ni de Postman. Los controladores no conocen los detalles internos del Trie o de la Tabla Hash. El servicio actúa como intermediario y coordina el funcionamiento completo.

En segundo lugar, se utilizaron Generics, Comparator e interfaces funcionales para evitar estructuras rígidas. Esto permite modificar criterios de ordenamiento sin alterar la implementación del heap o del Trie.

También se aplicaron validaciones para evitar palabras vacías, significados vacíos, duplicados o búsquedas inválidas. Estas validaciones ayudan a mantener el estado consistente del diccionario.

La persistencia automática mediante CSV fue otra decisión importante. El sistema guarda los cambios relevantes sin que el usuario tenga que hacerlo manualmente después de cada operación.

Buena práctica	Manifestación concreta en el proyecto
Separación de responsabilidades	Controladores, servicio, modelos, estructuras y utilidades se ubican en paquetes distintos.
Programación genérica	Trie<T>, CustomHashTable<K,V>, CustomPriorityQueue<T> y CustomArrayList<T> usan parámetros genéricos.
Delegación de criterio	Comparator<T> permite cambiar entre orden alfabético y frecuencia.
Inyección de comportamiento	ExtractorClave permite indicar al Trie cómo obtener la clave textual.
Evitar colecciones prohibidas	Se implementaron lista propia, hash propia y heap propio para la lógica principal.
Persistencia desacoplada	CsvManager se encarga de leer y escribir el archivo CSV.
Validación de entradas	El servicio valida texto vacío, duplicados e IDs inexistentes.
Arquitectura REST clara	Los endpoints siguen métodos HTTP coherentes: GET, POST, PUT y DELETE.
Preparación para defensa	Los comentarios en el código explican la finalidad de cada línea relevante.
Consistencia de estado	Cada cambio se refleja en Trie, Hash, lista y CSV.

10. Manual de funcionalidad del sistema

Esta sección describe el uso del sistema desde la perspectiva del usuario que realiza pruebas en Postman. El objetivo no es solamente indicar qué endpoint se ejecuta, sino explicar qué se está comprobando, qué datos deben enviarse, qué respuesta se espera y qué estructura participa internamente.

Antes de iniciar las pruebas, el proyecto debe estar ejecutándose desde IntelliJ. En la consola debe aparecer un mensaje similar a Tomcat started on port 8080. Esto confirma que Spring Boot levantó el servidor y que el API está disponible en `http://localhost:8080`.

10.1 Preparación inicial en Postman

1. Abrir Postman.
2. Crear una nueva solicitud.
3. Seleccionar el método HTTP correspondiente.
4. Escribir la URL iniciando con `http://localhost:8080`.
5. Cuando el endpoint requiera cuerpo, seleccionar Body, raw y JSON.
6. Presionar Send y revisar la respuesta.

10.2 Crear una palabra nueva

Esta prueba verifica que el sistema pueda almacenar una palabra nueva, asignarle un ID secuencial, establecer frecuencia inicial en 1, insertarla en la Tabla Hash por palabra, en la Tabla Hash por ID, en el Trie y posteriormente guardar el estado en el CSV.

```
POST http://localhost:8080/palabra
```

```
Body -> raw -> JSON
{
  "palabra": "hola",
  "significado": "saludo"
}
```

La respuesta esperada debe contener id, palabra, significado y frecuencia. Si es la primera palabra, normalmente el id será 1 y la frecuencia será 1.

10.3 Agregar varias palabras de prueba

Para demostrar correctamente prefijos, comodines y ordenamiento, conviene agregar varias palabras relacionadas. Por ejemplo: hola, hilo, halo, honda, hogar, hoja, horno, hotel y helado. Estas palabras permiten probar patrones como `ho` y `h*l*`.

```
{ "palabra": "hilo", "significado": "fibra delgada" }
{ "palabra": "halo", "significado": "circulo luminoso" }
{ "palabra": "honda", "significado": "automovil" }
{ "palabra": "hogar", "significado": "lugar donde vive una familia" }
```

10.4 Buscar por ID

Esta prueba comprueba el uso de la Tabla Hash por ID. Cuando el valor en la URL es numérico, el servicio interpreta que se debe buscar por identificador.

```
GET http://localhost:8080/palabra/1
```

Al encontrar la palabra, el sistema aumenta su frecuencia y guarda el cambio en el CSV. Esto evidencia el conteo de frecuencia de uso.

10.5 Buscar por coincidencia exacta

Esta prueba comprueba el uso de la Tabla Hash por palabra. Cuando el valor en la URL no es numérico, el servicio lo interpreta como texto y busca una coincidencia exacta.

```
GET http://localhost:8080/palabra/hola
```

La frecuencia vuelve a aumentar porque la palabra fue consultada. Esta operación demuestra búsqueda exacta y actualización de frecuencia.

10.6 Buscar por prefijo

Esta prueba utiliza el Trie. El prefijo ho permite recuperar palabras que inician con esas letras, como hola, honda, hogar, hoja, horno u hotel. Además, se puede limitar la cantidad de resultados y ordenar alfabéticamente o por frecuencia.

```
GET http://localhost:8080/prefijo/ho?limite=2&orden=desc&ordenarPor=alfabeto
```

El parámetro limite=2 indica que solo se desean dos resultados. orden=desc indica orden descendente. ordenarPor=alfabeto indica que se usa comparación alfabética.

10.7 Buscar por comodín

Esta prueba utiliza el Trie con búsqueda recursiva. El patrón h*1* permite buscar palabras que empiezan con h, luego tienen alguna secuencia de caracteres, después una l y finalmente otra posible secuencia.

```
GET http://localhost:8080/comodin/h*1*?limite=3&ordenarPor=frecuencia&orden=asc
```

Esta prueba demuestra comodín, límite de resultados y ordenamiento por frecuencia. Internamente, el Trie encuentra coincidencias y el Heap ordena los resultados.

10.8 Actualizar palabra

La actualización modifica una palabra existente usando su ID. Si cambia el texto de la palabra, el sistema debe eliminar la versión anterior de la Tabla Hash por palabra y del Trie, y luego insertar la versión actualizada.

```
PUT http://localhost:8080/palabra
```

```
Body -> raw -> JSON
```

```
{  
  "id": 1,  
  "palabra": "adios",  
  "significado": "despedida",  
  "frecuencia": 5  
}
```

Después de actualizar, se puede comprobar con GET /palabra/adios. La palabra anterior ya no debería encontrarse si el texto fue cambiado.

10.9 Eliminar palabra

La eliminación comprueba que el sistema elimine la palabra de todas sus estructuras internas y del CSV.

```
DELETE http://localhost:8080/palabra/1
```

Después se debe intentar buscar la misma palabra por ID. El sistema debe responder que no fue encontrada.

10.10 Exportar CSV

La exportación guarda el estado actual del diccionario dentro del archivo diccionario.csv. Esta prueba evidencia que los datos pueden conservarse fuera de la memoria del programa.

```
GET http://localhost:8080/csv/exportar
```

10.11 Importar CSV

La importación recarga el diccionario desde el archivo CSV. Para probarla, se puede editar manualmente `diccionario.csv` y agregar una línea nueva. Luego se ejecuta el endpoint de importación y se busca la palabra agregada.

```
POST http://localhost:8080/csv/importar
```

Ejemplo de línea en CSV:

```
10,perro,animal domestico,2
```

Después de importar, se verifica con `GET /palabra/perro`. Si aparece, significa que la palabra fue cargada desde archivo y reconstruida en las estructuras internas.

10.12 Comprobación de persistencia entre ejecuciones

1. Crear varias palabras desde Postman.
2. Verificar que aparezcan en `diccionario.csv`.
3. Detener el proyecto en IntelliJ.
4. Volver a ejecutar la aplicación.
5. Buscar una palabra creada anteriormente.
6. Confirmar que sigue existiendo.

Esta prueba demuestra que el sistema no depende únicamente de memoria temporal. El archivo CSV permite reconstruir el estado cada vez que el programa inicia.

10.13 Qué debe comprender el usuario al utilizar el programa

- Que POST crea palabras nuevas y las inserta en todas las estructuras.
- Que `GET /palabra/{valor}` puede buscar por ID o por texto exacto.
- Que las búsquedas exactas aumentan la frecuencia de uso.
- Que el Trie se utiliza para prefijos y comodines.
- Que el Heap se utiliza para ordenar y limitar resultados.
- Que la Tabla Hash permite búsquedas exactas eficientes.
- Que el CSV conserva los datos entre ejecuciones.
- Que los parámetros `limite`, `orden` y `ordenarPor` cambian la forma en que se presentan los resultados.

11. Conclusiones generales

Se concluye que el proyecto permitió integrar de manera coherente varias estructuras de datos avanzadas dentro de un sistema funcional. El problema del diccionario no podía resolverse de forma óptima con una sola estructura, ya que cada operación requería una estrategia distinta. La búsqueda exacta se benefició de la Tabla Hash, el autocompletado del Trie y el ordenamiento dinámico del Heap.

El Trie demostró ser la estructura más adecuada para búsquedas por prefijo y comodín. Su organización por caracteres permite recorrer únicamente las rutas relevantes, lo cual es más eficiente que revisar todas las palabras una por una cuando se trabaja con prefijos específicos.

La Tabla Hash permitió resolver búsquedas exactas por palabra y por ID de forma eficiente en el caso promedio. El manejo de colisiones por encadenamiento fortaleció la implementación porque contempló situaciones en las que varias claves pueden caer en el mismo bucket.

La Cola de Prioridad basada en Heap permitió ordenar resultados de manera flexible sin depender de PriorityQueue de Java. El uso de Comparator hizo posible cambiar el criterio de prioridad entre frecuencia y alfabeto, cumpliendo con el requisito de delegación e inyección de comportamiento.

La persistencia mediante CSV permitió conservar el estado del diccionario entre ejecuciones sin utilizar una base de datos externa. Esta decisión fue adecuada para el alcance académico del proyecto porque permite observar directamente los datos almacenados y facilita la demostración durante la defensa.

Desde la perspectiva de programación, el uso de Generics, lambdas, Comparator e interfaces funcionales mejoró la reutilización del código y evitó que las estructuras dependieran de una única clase o de un único criterio rígido. Esto evidencia una solución más flexible y mejor diseñada.

Por último, se puede afirmar que el proyecto representa una integración completa entre teoría de estructuras de datos y desarrollo de software. No solo se implementaron algoritmos, sino que se construyó un API verificable con Postman, se documentó su funcionamiento y se analizó formalmente su complejidad temporal y espacial.